สรุปและวิเคราะห์ผล Knime ETL Benchmark

Created	@March 4, 2026 8:18 AM
Tags	

จากการทดสอบและจับเวลาได้ไฟล์ที่สามารถสรุปเป็นตารางได้ดังนี้:

dataset	extract_sec	transform_sec	load_sec
10k	0.238	1.045	1.868
100k	0.759	8.221	41.728
1m	19.215	141.213	469.997

จากผลการจับเวลาที่บันทึกในตาราง สรุปได้ว่า KNIME ETL Pipeline ใช้เวลารวม 3.151 วินาที, 50.708 วินาที และ 630.425 วินาที สำหรับ Dataset ขนาด 10,000, 100,000 และ 1,000,000 แถว ตามลำดับ โดยรายละเอียดของแต่ละ Phase แสดงในกราฟที่ 1 ถึง 3 ด้านล่าง

1. KNIME : การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล

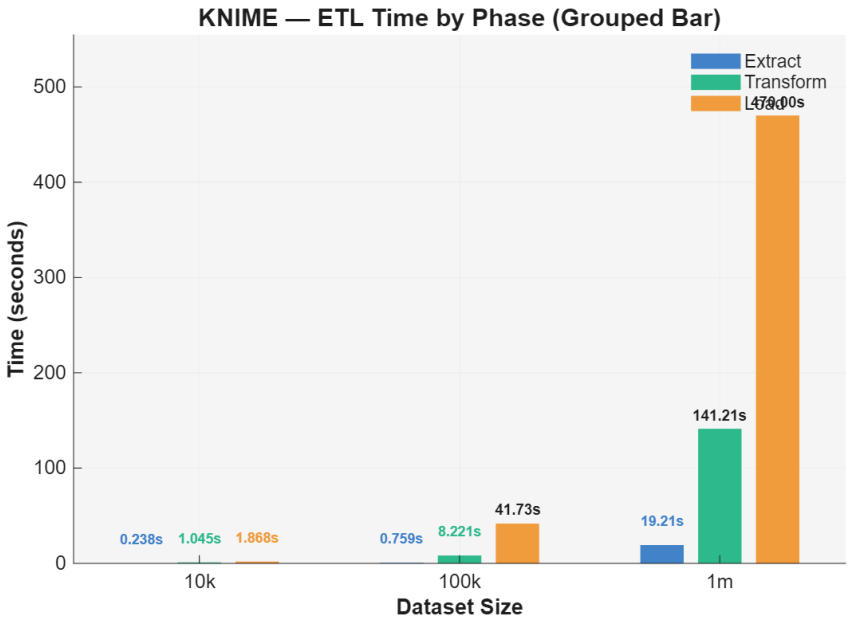

Fig 1 : Knime - การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล

กราฟนี้แสดงเวลาของแต่ละ Phase แยกออกจากกันให้เห็นชัดเจนในแต่ละขนาด Dataset จะเห็นว่าแท่งสีส้ม (Load) สูงกว่าแท่งอื่นในทุกขนาด และที่ 1,000,000 แถว Load ใช้เวลา

470.00 วินาที ซึ่งสูงกว่า Transform (141.21s) และ Extract (19.21s) อย่างชัดเจน เอกสาร NodePit **[3]** ระบุไว้ในหัวข้อ Batch size ของ DB Writer Node ว่า "Entering 1 disables batch processing. This way only one row is processed at a time." หมายความว่าข้อมูลถูกส่งไปยังฐานข้อมูลทีละ 1 แถว NodePit จากผลการทดสอบพบว่าทำให้เวลา Load เพิ่มขึ้นเร็วกว่า Extract และ Transform อย่างเห็นได้ชัดเจนเมื่อขนาด Dataset ใหญ่ขึ้น

2. KNIME : สัดส่วนของแต่ละ Phase ต่อเวลารวม

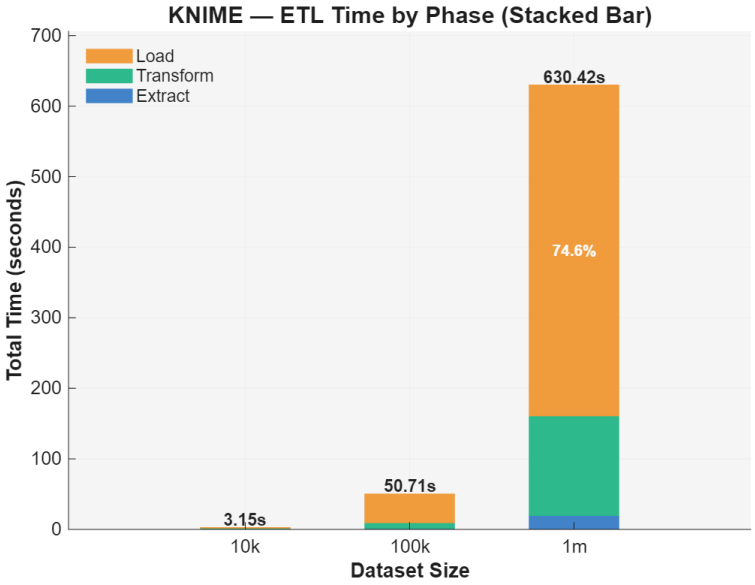

Fig 2: Knime - สัดส่วนของแต่ละ Phase ต่อเวลารวม

กราฟนี้แสดงเวลารวมทั้งหมดของแต่ละขนาดชุดข้อมูล โดยแบ่งสัดส่วนของแต่ละ Phase ให้เห็นในแท่งเดียวกัน เวลารวมอยู่ที่ 3.151 วินาที, 50.708 วินาที และ 630.425 วินาที ตามลำดับ ที่ 1,000,000 แถว ขั้นตอนการโหลดข้อมูล (สีส้ม) กินสัดส่วนถึง 74.6% ของเวลาทั้งหมด สาเหตุที่ Load Phase ใช้เวลานานเพราะว่ามีการใช้ DB Writer Node ซึ่งประมวลผลทีละ 1 แถว ซึ่งเป็นการปิดการทำงานแบบ Batch เมื่อ KNIME ส่งข้อมูลทีละแถว MySQL จำเป็นต้องบันทึกข้อมูลลงดิสก์ทุกครั้งที่มีการยืนยันคำสั่ง และความเร็วของอุปกรณ์จัดเก็บข้อมูลเป็นตัวกำหนดขีดจำกัดของจำนวนคำสั่งสูงสุดที่ทำได้ต่อวินาที **[4]** ซึ่งการทดสอบนี้พบว่าขั้นตอนการโหลดข้อมูลของ KNIME ใช้เวลามากกว่า SSIS อย่างมีนัยสำคัญในทุกขนาดชุดข้อมูล สาเหตุอาจมาจาก Driver และ Connection Protocol ระหว่าง ODBC Destination กับ JDBC

3. KNIME : อัตราการขยายตัวของแต่ละ Phase (Scalability)

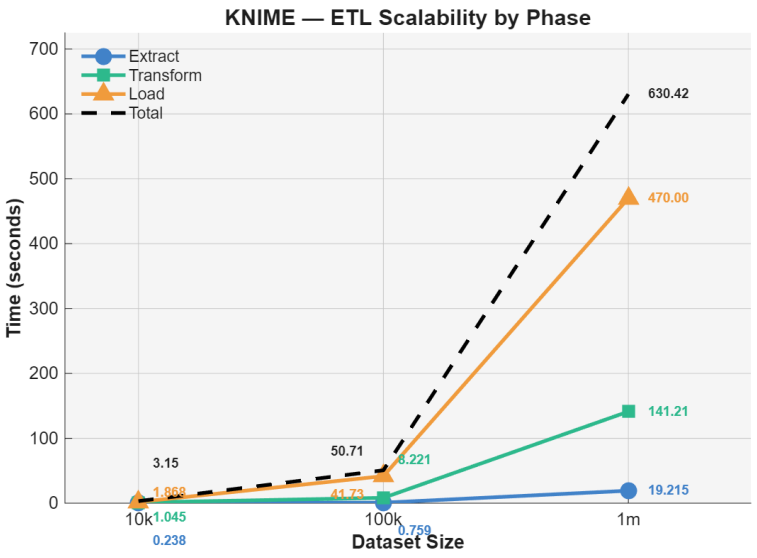

Fig 3 : Knime - อัตราการขยายตัวของแต่ละ Phase

กราฟนี้แสดงให้เห็นว่าแต่ละ Phase ขยายตัวเร็วแค่ไหนเมื่อข้อมูลโตขึ้น เส้นสีส้ม (Load) เป็นเส้นที่ชันที่สุด โดยพุ่งจาก 1.868s ไปถึง 470.00s เมื่อข้อมูลโต 100 เท่า ในขณะที่เส้นสีน้ำเงิน (Extract) ราบเรียบที่สุด สาเหตุที่ Load ชันมากเพราะ JDBC Driver ต้องส่งคำสั่ง COMMIT และ BEGIN TRANSACTION ทุกครั้งที่ส่งข้อมูล**[5]** ดังนั้นเมื่อแถวข้อมูลเพิ่มขึ้น ภาระของ Load จึงโตเร็วกว่า Phase อื่นอย่างเห็นได้ชัด

References

[3]"DB writer," *NodePit*. [Online]. Available: https://nodepit.com/node/org.knime.database.node.io.write.DBWriteNodeFactory.

[4]*Mysql.com*. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/optimizing-innodb-transaction-management.html.

[5]"JDBC driver application performance optimization tutorial," *Progress.com*. [Online]. Available: https://www.progress.com/tutorials/jdbc/designing-performance.